Assignment5: Exploring Data-Level Parallelism (DLP) in Modern Computing

Haeri Kyoung

University of the Cumberlands

MSCS-531-A01 – Computer Architecture and Design

Professor Jay Thom

June 4, 2025

Part 1: Understanding Data-Level Parallelism

Introduction to DLP

Data-level parallelism, or DLP, refers to the ability of a system to apply the same operation to multiple pieces of data at once. It's especially useful in programs that need to process large amounts of similar data using the same steps. Rather than handling one item at a time in a loop, DLP makes it possible to work on entire sets of data in parallel, which can significantly boost performance.

This is quite different from instruction-level parallelism, which focuses on running multiple different instructions at the same time. While instruction-level parallelism is limited by how instructions depend on each other, DLP works best in cases where the same action needs to be repeated many times. Because of this, DLP tends to be more predictable and easier to optimize in data-heavy tasks.

DLP plays a key role in areas like multimedia, scientific computing, and machine learning. For example, image filters, video processing, and audio enhancements often involve applying the same math to every pixel or sample. Scientific simulations frequently use large matrices and arrays that are ideal for data-parallel operations. In machine learning, both training and inference rely on repeating the same calculations across thousands or millions of values, which makes DLP a natural fit.

To take advantage of DLP, hardware needs to be designed with specific features. Vector architectures use registers that hold several values and allow one instruction to operate on all of them at once. This cuts down on the number of instructions needed and improves overall efficiency. Another common approach is SIMD, which stands for single instruction multiple data. These in-

struction sets are built into modern CPUs and let each core apply a single operation to multiple values at the same time.

By enabling processors to handle more work in fewer steps, DLP has become a crucial part of modern computing. It helps systems run faster without simply increasing clock speed or adding more cores, and it provides a path forward for performance gains in data-intensive applications.

Part 2: Exploring DLP Architectures

Vector Architectures

Vector architectures are designed to perform operations on multiple data points simultaneously using a single instruction. This parallel execution approach is known as Single Instruction, Multiple Data (SIMD). Unlike scalar architectures that operate on a single element at a time, vector processors allow computations across arrays or vectors of data to happen concurrently, making them ideal for high-performance tasks like scientific simulations, multimedia processing, and machine learning.

To demonstrate this concept, I implemented a simple vector addition task using SSE (Streaming SIMD Extensions) intrinsics in C. SSE uses 128-bit registers capable of holding four 32-bit floating-point numbers. The code loads two float arrays into SIMD registers, performs the addition in parallel, and stores the result in a third array. This reduces the number of instructions required compared to scalar addition.

This screenshot shows the C code implementing SIMD vector addition using SSE intrinsics.

```
  GNU nano 7.2  vector_add_sse.c
#include <xmmintrin.h>
#include <stdio.h>

int main() {
    float A[4] = {1.0, 2.0, 3.0, 4.0};
    float B[4] = {4.0, 3.0, 2.0, 1.0};
    float C[4];

    __m128 vecA = _mm_loadu_ps(A);
    __m128 vecB = _mm_loadu_ps(B);
    __m128 vecC = _mm_add_ps(vecA, vecB);

    _mm_storeu_ps(C, vecC);

    for (int i = 0; i < 4; i++) {
        printf("%.1f ", C[i]);
    }

    return 0;
}
```

Figure 1: Source Code (vector_add_sse.c)

The following command was used to compile the code with SSE enabled:

```
gcc -msse -O2 -o vector_add_sse vector_add_sse.c
```

The screenshot below confirms successful compilation without errors. Running the compiled binary produces the expected result of four 5.0 values, confirming the correctness of the SIMD operation.

```
root@gem5_dev:/gem5# gcc -msse -O2 -o vector_add_sse vector_add_sse.c
./vector_add_sse
5.0 5.0 5.0 5.0 root@gem5_dev:/gem5# nano vector_add_sse
```

Figure 2: Compilation Command and Program Output

Running the compiled binary produces the expected result of four 5.0 values, confirming the correctness of the SIMD operation.

This simulation confirms the effectiveness of SIMD for simple, repetitive tasks like array addition. In a scalar implementation, four addition instructions would be required; with SIMD, a single instruction performs all four additions at once, offering a clear performance advantage.

Discussion

Vector architectures offer significant performance benefits in data-parallel tasks. They reduce instruction count, accelerate execution, and increase throughput. This makes them particularly useful in domains like digital signal processing, video encoding, and deep learning inference. However, vector processing is not without challenges. Many SIMD instructions require data to be aligned in memory, and misaligned accesses can lead to slowdowns or faults. Moreover, not all workloads are suitable for vectorization, particularly those involving complex branching or irregular memory access patterns.

Another limitation is hardware dependence. SIMD code written for SSE may not be compatible with newer or different instruction sets like AVX, NEON, or SVE. Developers must often write multiple code paths or rely on libraries that abstract away the hardware differences.

Despite these limitations, vector architectures remain a critical part of modern processors. They provide an efficient way to leverage data-level parallelism and are essential in achieving high performance without increasing clock speeds or energy consumption.

SIMD Instruction Set Extensions

To understand the performance benefits of SIMD (Single Instruction, Multiple Data) instructions, I implemented a simple program that adds two arrays of floating-point numbers. I wrote both a scalar version using standard for-loops and a SIMD version using SSE intrinsics.

For the SIMD version, I used the _mm_loadu_ps intrinsic to load four floats into a 128-bit SSE register, performed vector addition using _mm_add_ps, and stored the result using _mm_storeu_ps. This approach allows the processor to add four elements at once using a single instruction, which is much faster than processing each element individually.

Both programs were compiled and run inside the /gem5/simd_test directory. I used the following commands:

```
gcc -O2 -o scalar_add scalar_add.c
gcc -msse -O2 -o simd_add_sse simd_add_sse.c
./scalar_add
./simd_add_sse
```

Both versions correctly printed:

```
root@gem5_dev:/gem5/simd_test# gcc -O2 -o scalar_add scalar_add.c
gcc -msse -O2 -o simd_add_sse simd_add_sse.c
root@gem5_dev:/gem5/simd_test# ./scalar_add
./simd_add_sse
5.0 5.0 5.0 5.0
5.0 5.0 5.0 5.0
```

Figure 3. Compilation Command and Program Output

This confirms that the results were accurate for both approaches.

Comparison

Although the output of the scalar and SIMD versions is the same, the way they compute the results is fundamentally different. The scalar version processes each array element one at a time, while the SIMD version handles four elements in parallel. For small arrays like this, you won't notice a big speed difference, but the advantage of SIMD becomes clear when working with large data sets, such as in image processing or scientific computations.

Using SIMD reduces the number of instructions needed and improves performance significantly when there's a lot of data to process. One challenge is ensuring the data is properly aligned and that the processor supports the SIMD instructions being used. In this case, I chose SSE instead of AVX due to better compatibility within the Docker container.

This exercise demonstrates how SIMD can be used to accelerate simple data-parallel tasks. With just a few changes to the code, we can leverage SIMD intrinsics for better performance. It's a practical and effective optimization technique that plays a big role in high-performance applications today.

Part 3: GPUs and DLP

Architecture and Parallelism in GPUs

Graphics Processing Units (GPUs) were originally built to render images and handle visual effects, but their structure has made them incredibly powerful for general-purpose computing, especially when the problem involves performing the same operation on large sets of data. This is known as data-level parallelism (DLP), and it plays to the strengths of the GPU's design.

GPUs contain thousands of smaller, more energy-efficient cores compared to the relatively fewer, more complex cores in CPUs. These cores operate under a model called SIMT, or Single Instruction, Multiple Threads. This means a single instruction is executed by multiple threads in parallel. Threads are grouped into units called warps (typically 32 threads), and multiple warps form a thread block. The hardware schedules and executes warps in parallel, maximizing throughput as long as the threads within a warp follow the same execution path.

Another reason GPUs are effective for parallel tasks is their well-organized memory hierarchy. There is global memory shared across all threads, shared memory used within thread blocks, and local memory used per thread. This structure allows efficient memory usage, although poor memory access patterns or diverging threads (where threads take different branches in the code) can lead to performance penalties.

In short, GPUs excel at running the same operation on massive arrays of data, which is why they are so commonly used for scientific computing, image processing, training AI models, and matrix operations.

Matrix Multiplication on the GPU

For this case study, matrix multiplication was chosen because it is a fundamental operation in many domains, including graphics, physics simulations, and machine learning. The task involves multiplying two large matrices by computing the dot product of rows and columns repeatedly, which makes it a great fit for parallel processing.

Since the Docker container environment used for this assignment doesn't have GPU hardware support (and CUDA could not be installed successfully), a simulation was done using NumPy to perform CPU-based matrix multiplication. Although this doesn't use GPU acceleration, it provides a baseline for how long it takes to run on the CPU. The code used was:

```
 GNU nano 7.2              gpu_matrix_mul_numpy.py
import numpy as np
import time

# Matrix size
N = 1024

# Create random matrices
A = np.random.rand(N, N).astype(np.float32)
B = np.random.rand(N, N).astype(np.float32)

# Matrix multiplication using NumPy (which uses BLAS under the hood)
start = time.time()
C = np.dot(A, B)
end = time.time()

print("Matrix multiplication completed.")
print(f"Elapsed time: {end - start:.4f} seconds")
```

Figure 4. Python Code Used for Matrix Multiplication

The output showed the CPU took approximately 0.0368 seconds to multiply two 1024×1024 matrices. While this seems fast, a GPU implementation using CuPy or PyTorch with CUDA could achieve several times faster performance, especially as matrix sizes increase.

```
(venv) root@gem5_dev:/gem5/MSCS531_Assignment5# python3 gpu_matrix_mul_numpy.py
Matrix multiplication completed.
Elapsed time: 0.0368 seconds
```

Figure 5. gpu_matrix_mul_numpy,py Output

Challenges and Optimization Techniques

Optimizing GPU performance goes beyond just moving the code from CPU to GPU. There are several techniques developers use to fully harness GPU parallelism:

- **Memory Coalescing**: Ensuring threads in a warp access consecutive memory locations to avoid delays.
- **Avoiding Warp Divergence**: Making sure threads within a warp take the same execution path to prevent serialization.
- **Using Shared Memory**: Reducing global memory access by loading reusable data into faster shared memory.
- **Occupancy Tuning**: Adjusting the number of threads and blocks to maximize utilization of GPU resources without exceeding limits.

GPU performance can also be affected by PCIe data transfer bottlenecks, limited shared memory, or inefficient thread scheduling. Profiling tools like NVIDIA Nsight or CUDA Visual Profiler are often used to fine-tune GPU code.

Conclusion

GPUs are purpose-built for tasks involving high data parallelism, and their architecture is fundamentally different from CPUs. By using many simple cores and structured parallelism models like SIMT, GPUs handle large-scale computations more efficiently. While the actual GPU simulation couldn't be performed in this environment due to missing CUDA support, it's clear that tasks like matrix multiplication benefit significantly from GPU acceleration. As workloads continue to grow in scale and complexity, GPUs remain a key tool for achieving performance gains in modern computing.

Part 4: Loop-Level Parallelism and DLP in Software

Enhancing Loop-Level Parallelism

Loop-level parallelism is a fundamental technique in software optimization that allows individual iterations of a loop to be executed concurrently. This is especially useful for data-parallel tasks where each iteration operates independently. One common approach to implement loop-level parallelism is through OpenMP, a standard for parallel programming in C/C++ that provides simple compiler directives to parallelize loops.

In this task, we created two versions of a program to sum the elements of a large array: a **serial** version and a parallel version using OpenMP. The serial version computes the sum by iterating through the array in a single thread. The parallel version, on the other hand, uses the #pragma omp parallel for directive to split the iterations across multiple threads.

After compiling both versions with GCC and running them in our Docker environment, we measured the execution time using clock() from the C standard library.

Here is the command output showing the performance:

```
root@gem5_dev:/gem5/MSCS531_Assignment5/loop_parallelism# gcc -O2 -o loop_serial loop_serial.c
gcc -fopenmp -O2 -o loop_parallel loop_parallel.c
root@gem5_dev:/gem5/MSCS531_Assignment5/loop_parallelism# ./loop_serial
./loop_parallel
Elapsed time: 0.3863 seconds
Elapsed time (parallel): 0.0850 seconds
```

Figure 6. Compilation Command and the Output

The parallel implementation reduced execution time significantly, demonstrating how loop-level parallelism can improve performance when the workload is large and evenly distributed.

Reflection

Loop-level parallelism is one of the most accessible and impactful forms of parallelization in modern software development. It can be applied to many scenarios involving numerical computation, large datasets, and simulations. OpenMP makes this even more convenient by allowing developers to parallelize loops with minimal code changes.

However, applying loop-level parallelism effectively requires careful consideration of data dependencies. If loop iterations depend on the results of previous ones, parallelizing them could cause race conditions or incorrect outputs. It is also important to balance the workload among threads to avoid bottlenecks, especially on systems with many cores.

In this case, because each iteration was independent, OpenMP was able to improve performance without introducing synchronization issues. This illustrates how loop-level parallelism can help take full advantage of modern multicore processors when applied to the right problem.

Part 5: Reflection and Emerging Trends

Performance, Complexity, and Energy Efficiency

Designing systems that leverage Data-Level Parallelism (DLP) always involves balancing three major factors: performance, complexity, and energy efficiency. While DLP techniques such as

SIMD, vector processing, and GPU acceleration can dramatically improve performance for data-parallel tasks, this gain often comes at the cost of increased architectural complexity and higher energy consumption.

For example, widening vector registers or adding more execution units can speed up computations but also makes the processor more complex to design and verify. These hardware improvements may lead to diminishing returns if the workload does not scale well or if the software isn't optimized to take full advantage of parallelism.

Energy efficiency is another major concern, especially in mobile and embedded environments. Techniques like power gating and dynamic voltage scaling help reduce energy consumption, but they require extra hardware support and sophisticated runtime management. Modern processors must constantly balance throughput and power usage to meet the diverse needs of data centers, edge devices, and consumer electronics.

In short, designing for DLP means navigating trade-offs: you might gain speed at the cost of increased design effort or power drain. The best systems are those that find a smart balance depending on the target workload and deployment environment.

Emerging Trends and Challenges

We are currently witnessing a rapid evolution in DLP-driven processor design. Here are a few key trends:

- **GPU Innovations**: GPUs are becoming more specialized and programmable, with support for mixed-precision operations and larger memory bandwidths tailored for AI and scientific workloads.

- **AI Accelerators**: Custom chips like Google's TPU and Apple's Neural Engine are pushing the boundaries of DLP. These accelerators are optimized for matrix and tensor operations, bringing parallelism to machine learning inference at massive scale.
- **Heterogeneous Computing**: Modern systems often combine CPUs, GPUs, and domain-specific accelerators to exploit parallelism at multiple levels. Managing these systems effectively is a growing challenge.
- **Better Toolchains**: Advances in compilers and frameworks (like OpenCL and CUDA) are making it easier to express and manage DLP in code, abstracting away hardware details.

However, as systems grow more complex, so do the challenges. Memory bandwidth often becomes a bottleneck, and coordinating parallel workloads across different compute units adds runtime overhead. Additionally, designing energy-efficient systems is becoming harder as performance demands grow. Architectural decisions must now consider not just raw speed, but power envelopes, thermal limits, and sustainability goals.

In the future, we can expect continued innovation in software and hardware co-design, tighter integration of memory and compute, and more intelligent systems that dynamically adapt how they execute parallel tasks.

Submission Information

GitHub Repository:

https://github.com/hkyoung38554/MSCS531_Assignment5